

CSE 6220/CX 4220

Introduction to HPC

Lecture 10: MPI Collective Communication

Helen Xu

hxu615@gatech.edu



Georgia Tech College of Computing
School of Computational
Science and Engineering

Slides from UC Berkeley 267, Srinivas Aluru

Recap: The Simplest MPI Send Command

```
int MPI_Send(void *buf, int count,  
             MPI_Datatype datatype,  
             int dest, int tag,  
             MPI_Comm comm)
```

Returns only when communication is finished

This **blocking send function** returns when the data has been delivered to the system and the buffer can be reused. The message may not have been received by the destination process.

Recap: The Simplest MPI Receive Command

```
int MPI_Recv(void *buf, int count  
             MPI_Datatype datatype,  
             int source, int tag,  
             MPI_Comm comm,  
             MPI_Status *status);
```

- This **blocking receive function** waits until a matching message is received from the system so that the buffer contains the incoming message.
- Match of data type, source process (or MPI_ANY_SOURCE), message tag (or MPI_ANY_TAG).
- Receiving fewer datatype elements than count is ok, but receiving more is an error.

Example Send / Recv

```
#include <mpi.h>
#include <iostream>

int main(int argc , char* argv []) {
    int rank;
    MPI_Init (&argc , &argv);
    MPI_Comm_rank (MPI_COMM_WORLD , &rank );
    const int N = 32;
    int tab[N];
    if (rank == 0) {
        tab[N - 1] = 13;
        MPI_Send(tab, N, MPI_INT,
                 1, 111, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Status stat;
        MPI_Recv(tab, N, MPI_INT,
                 0, 111, MPI_COMM_WORLD, &stat);
        std::cout << tab[N - 1] << std::endl;
    }
    return MPI_Finalize();
}
```

Send N integers from rank 0 to rank 1.

Blocks till everything is sent.

Receive N integers.

Blocks till everything is received.

Output: 13

MPI Tags

```
int MPI_Send(void *buf, int count,  
             MPI_Datatype datatype,  
             int dest, int tag,  
             MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count  
            MPI_Datatype datatype,  
            int source, int tag,  
            MPI_Comm comm,  
            MPI_Status *status);
```

Messages are sent with an accompanying user-defined integer tag, to assist the receiving process in identifying the message.

Messages can be screened at the receiving end by specifying a specific tag, or not screened by specifying `MPI_ANY_TAG` as the tag in a receive.

Message Matching

- Each send operation **must be matched** by a corresponding receive operation
- Messages are matched via their **envelope**:
 - communicator
 - source rank
 - tag
 - **NOT** by type or size
- Thus: for a receive to succeed, a message needs to match the communicator, tag, and rank

Blocking:

- An MPI_Recv operation **blocks** until a matching message is received
- An MPI_Send operation **may block** until the message has been received by the destination process

Example: Message Matching

```
#include <mpi.h>
```

What's wrong?

```
int main(int argc , char* argv []) {  
    int rank;  
    MPI_Init(&argc , &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD , &rank);  
    int x;  
    if (rank == 0) {  
        x = 13;  
        MPI_Send(&x, 1, MPI_INT,  
                1, 111, MPI_COMM_WORLD);  
    } else if (rank == 1) {  
        MPI_Status stat;  
        MPI_Recv(&x, 1, MPI_INT,  
                0, 222, MPI_COMM_WORLD, &stat);  
    }  
    return MPI_Finalize();  
}
```

Tags are not matching
111 != 222

MPI program will
deadlock!

Example: Message Matching

```
#include <mpi.h>
int main(int argc, char* argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int x[16];
    float y[16];
    if (rank == 0) {
        x[13] = 13;
        y[13] = 0.13;
        MPI_Send(x, 16, MPI_INT,
                 1, 111, MPI_COMM_WORLD);
        MPI_Send(y, 16, MPI_FLOAT,
                 1, 222, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Status stat;
        MPI_Recv(y, 16, MPI_FLOAT,
                 0, 222, MPI_COMM_WORLD, &stat);
        MPI_Recv(x, 16, MPI_INT,
                 0, 111, MPI_COMM_WORLD, &stat);
    }
    return MPI_Finalize();
}
```

What's wrong?

How to fix it?

Blocks until message with tag 111 is received.

Blocks until message with tag 222 is sent.

MPI program will deadlock!

Example: Message Matching

```
#include <mpi.h>
int main(int argc, char* argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int x[16];
    float y[16];
    if (rank == 0) {
        x[13] = 13;
        y[13] = 0.13;
        MPI_Send(x, 16, MPI_INT,
                 1, 111, MPI_COMM_WORLD);
        MPI_Send(y, 16, MPI_FLOAT,
                 1, 222, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Status stat;
        MPI_Recv(y, 16, MPI_FLOAT,
                 0, 222, MPI_COMM_WORLD, &stat);
        MPI_Recv(x, 16, MPI_INT,
                 0, 111, MPI_COMM_WORLD, &stat);
    }
    return MPI_Finalize();
}
```

What's wrong?

How to fix it?

Reorder receives to x and y

Blocks until message with tag 111 is received.

Blocks until message with tag 222 is sent.

MPI program will deadlock!

Any

- Useful trick:
 - We can use any combination of **tag = MPI_ANY_TAG** and/or **source = MPI_ANY_SOURCE** in **MPI_Recv** to receive any message from any processor
 - We can use the **MPI_Status** structure to check envelope of the received message
- Example:

```
int x;  
MPI_Status stat;  
MPI_Recv(&x, 1, MPI_INT,  
        MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &stat);  
int source = stat.MPI_SOURCE;  
int tag = stat.MPI_TAG;
```

Example: any tag, any src

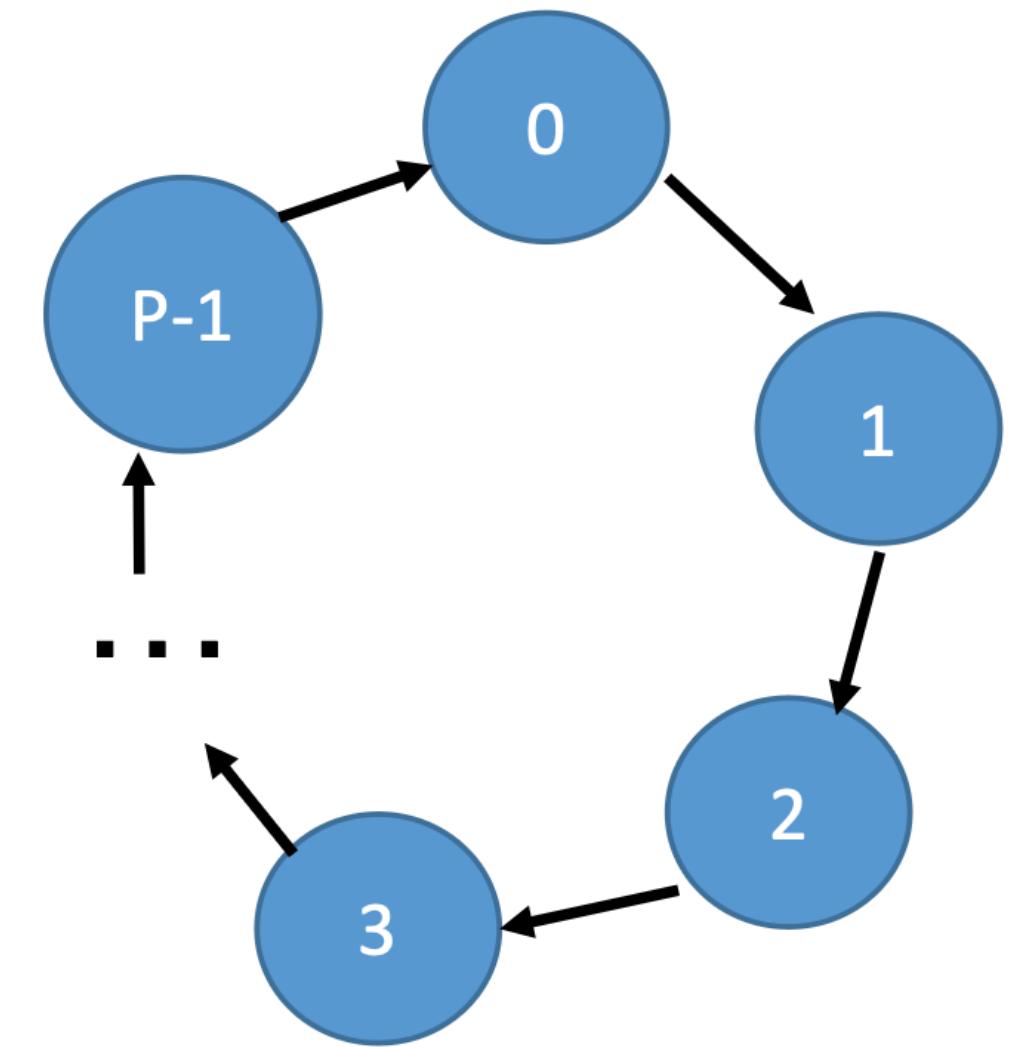
```
#include <mpi.h>
#include <iostream>

int main(int argc , char* argv []) {
    int rank , size;
    MPI_Init (&argc , &argv );
    MPI_Comm_rank (MPI_COMM_WORLD , &rank );
    MPI_Comm_size (MPI_COMM_WORLD , &size );
    int x;
    if (rank == 0) {
        MPI_Status stat;
        for (int i = 1; i < size; ++i) {
            MPI_Recv(&x, 1, MPI_INT,
                    MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD,
                    &stat);
            std::cout << "From: " << stat.MPI_SOURCE << " "
                      << "Tag: " << stat.MPI_TAG << std::endl;
        }
    } else {
        x = rank;
        MPI_Send(&x, 1, MPI_INT,
                0, size - rank, MPI_COMM_WORLD);
    }
    return MPI_Finalize();
}
```

Receives any one of the messages. No order guaranteed.

Cyclic Dependencies

- Let's say we want to send messages according to right shift permutation:
 - Processor i sends a message to $(i+1) \bmod p$



Example:

```
MPI_Send(&x, 1, MPI_INT,  
        (rank+1) % size, 13, MPI_COMM_WORLD);  
MPI_Recv(&y, 1, MPI_INT,  
        MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &stat);
```

What's wrong?

May block till message is received, MPI_Recv never called

Reverse?

```
MPI_Recv(&y, 1, MPI_INT,  
        MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &stat);  
MPI_Send(&x, 1, MPI_INT,  
        (rank+1) % size, 13, MPI_COMM_WORLD);
```

Same problem.

Non-blocking Communication

- Enter: Non-blocking Communication
 - **MPI_Isend** and **MPI_Irecv** will initiate transfers but **not block** until they succeed
 - Enables:
 - Dead-lock avoidance
 - Hiding latency by overlapping computation and communication
 - Same parameters as MPI_Send, MPI_Recv but additional output parameter:

```
int MPI_Isend(void* buf, int count, MPI_Datatype type,  
             int dest, int tag, MPI_Comm comm,  
             MPI_Request* req);
```

```
int MPI_Irecv(void* buf, int count, MPI_Datatype type,  
             int source, int tag, MPI_Comm comm,  
             MPI_Request* req);
```

MPI_Wait and MPI_Test

```
int MPI_Wait(MPI_Request* request,  
             MPI_Status* status);
```

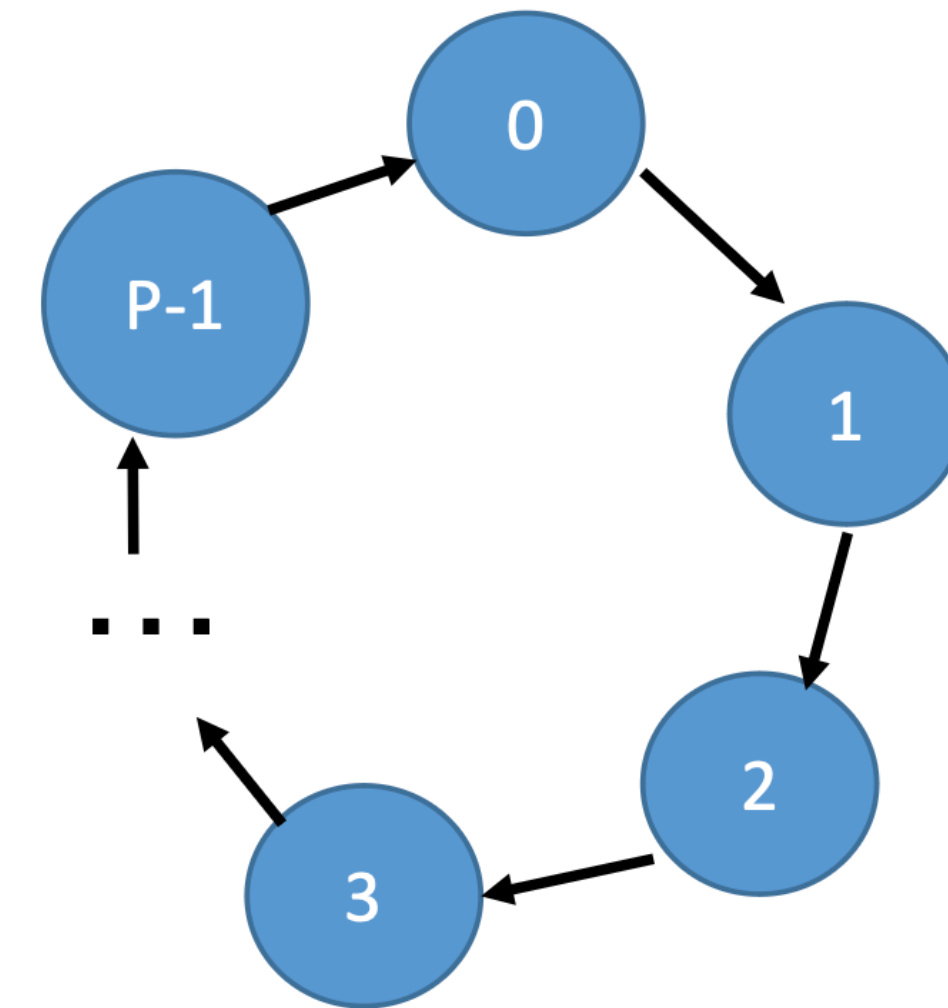
MPI_Wait **waits** for a non-blocking operation to complete and will block until the underlying operation is done.

```
int MPI_Test(MPI_Request* request,  
             int* flag,  
             MPI_Status* status);
```

MPI_Test checks if a non-blocking operation is complete at a given time. **It will not wait** for the underlying non-blocking operation to complete.

Cyclic Dependencies

- Let's say we want to send messages according to right shift permutation:
 - Processor i sends a message to $(i+1) \bmod p$



Example:

```
MPI_Send(&x, 1, MPI_INT,  
        (rank+1) % size, 13, MPI_COMM_WORLD);  
MPI_Recv(&y, 1, MPI_INT,  
        MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &stat);
```

What's wrong?

May block till message is received, MPI_Recv never called

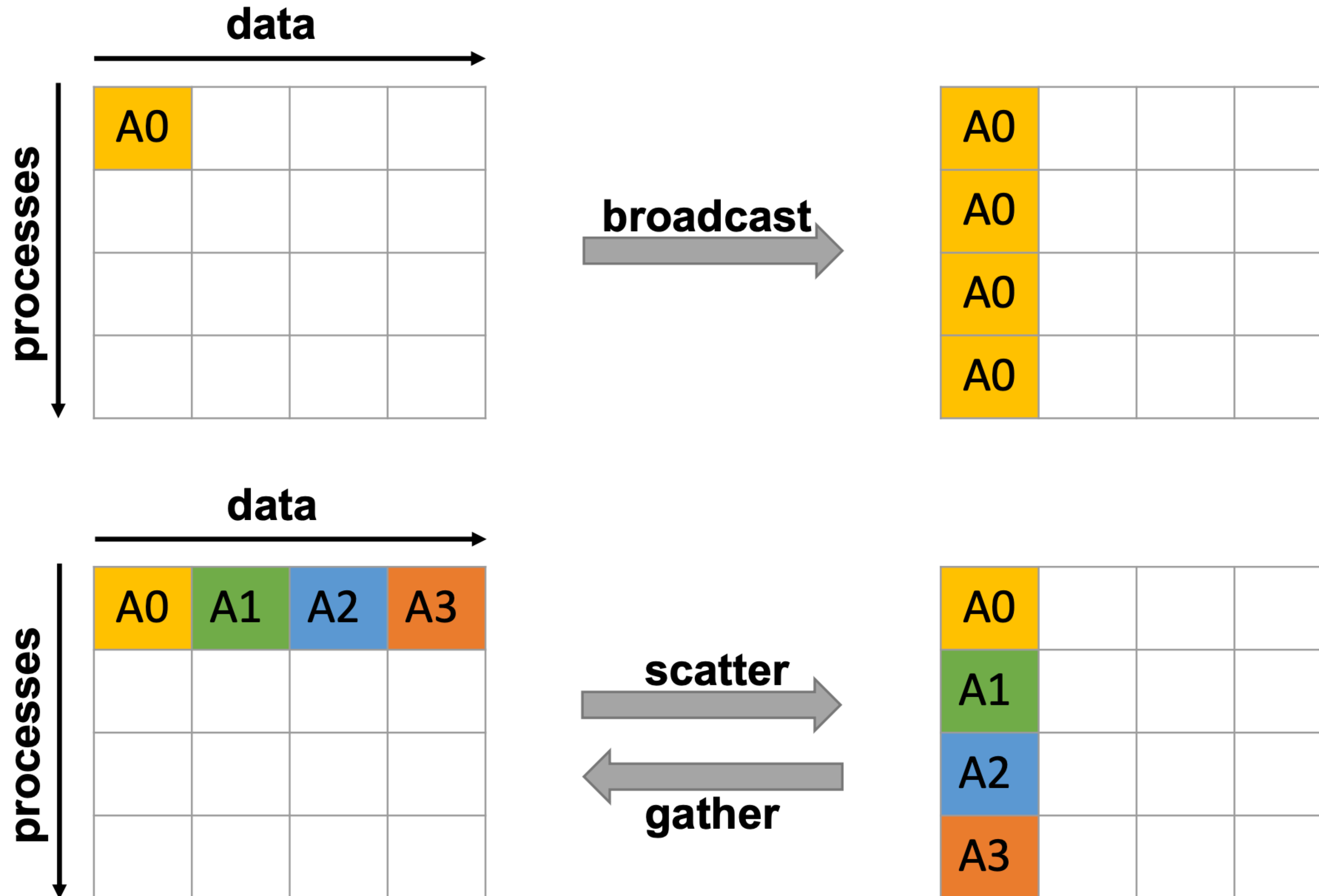
Using Non-blocking receive:

```
MPI_Request req;  
MPI_Irecv(&y, 1, MPI_INT,  
        MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &req);  
MPI_Send(&x, 1, MPI_INT,  
        (rank+1) % size, 13, MPI_COMM_WORLD);  
MPI_Wait(&req, &stat);
```

Correct!

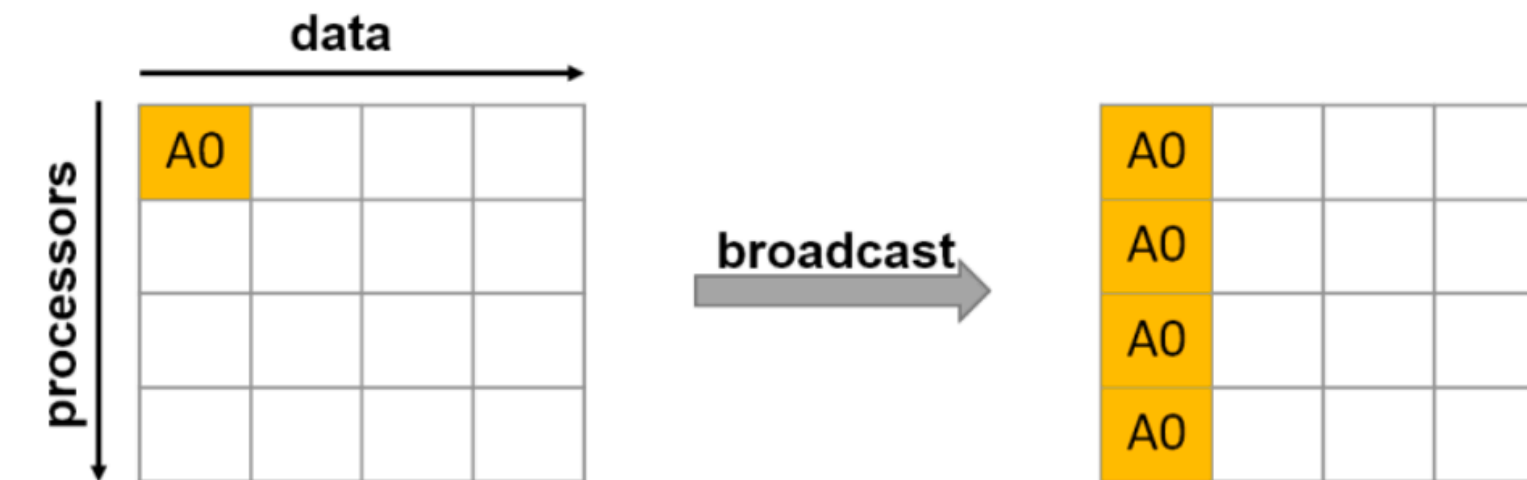
MPI Collectives

Broadcast, Scatter, Gather



Broadcast

Broadcast a message from one process(=root) to all other processes



```
int MPI_Bcast(void* buf, int count, MPI_Datatype type,  
              int root, MPI_Comm comm);
```

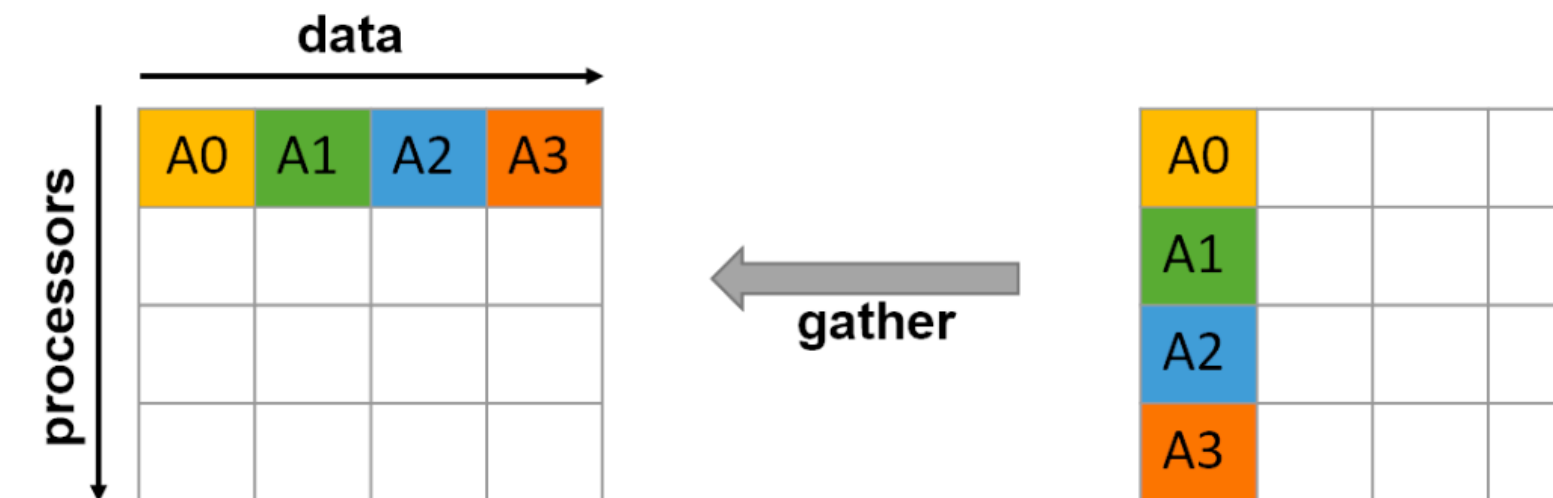
Example:

```
#include <mpi.h>  
#include <iostream>  
int main(int argc , char* argv []) {  
    int rank;  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);  
    double x = 0.0;  
    if (rank == 0) x = -13.13;  
    MPI_Bcast(&x, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);  
    std::cout << x << std::endl;  
    return MPI_Finalize ();  
}
```

Prints -13.13 on every process.

Gather

Gather data from all processes on one process(=root)



```
int MPI_Gather(void* sbuf, int scount, MPI_Datatype stype,  
              void* rbuf, int rcount, MPI_Datatype rtype,  
              int root, MPI_Comm comm);
```

```
#include <mpi.h>  
#include <iostream >  
#include <vector >  
int main(int argc, char* argv[]) {  
    int rank, size;  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
    MPI_Comm_size(MPI_COMM_WORLD, &size);  
    std::vector<int> buf;  
    if (rank == 0) buf.resize(size, -1);  
    MPI_Gather(&rank, 1, MPI_INT,  
              &buf[0], 1, MPI_INT, 0, MPI_COMM_WORLD);  
    if (rank == 0) std::cout << buf.back() << std::endl;  
    return MPI_Finalize();  
}
```

At this point, what is the content of the vector buf?

Scatter

Scatter data from one process to all processes



```
int MPI_Scatter(void* sbuf, int scount, MPI_Datatype stype,
               void* rbuf, int rcount, MPI_Datatype rtype,
               int root, MPI_Comm comm);
```

```
int main(int argc, char* argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    float x = 13;
    std::vector<float> buf;
    if (rank == 0) {
        buf.resize(size, -1);
        for (int i = 0; i < size; ++i) buf[i] = 1.0 / (1 + i);
    }
    MPI_Scatter(&buf[0], 1, MPI_FLOAT,
               &x, 1, MPI_FLOAT, 0, MPI_COMM_WORLD);
    std::cout << rank << " " << x << std::endl;
    return MPI_Finalize();
}
```

What is the content of the variable x on each process?

Scatterv

Array of send counts

```
int MPI_Scatterv(void* sbuf, int *scounts, int *displs,  
                MPI_Datatype stype,  
                void* rbuf, int rcount, MPI_Datatype rtype,  
                int root, MPI_Comm comm);
```

Array of start
indices

```
#include <mpi.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>
```

```
#define N 4
```

```
int main(int argc, char *argv[])  
{  
    int rank, size; // this process' rank, and the number of processes  
    int *sendcounts; // number of elements to send to each process  
    int *displs; // displacements where each segment begins  
    int sum=0; // Sum of counts. Used to calculate displacements  
    char rec_buf[100]; // buffer where the received data should be stored
```

```
    char data[N][N]={ // the data to be distributed  
        {'a','b','c','d'}, {'e','f','g','h'},  
        {'i','j','k','l'}, {'m','n','o','p'}  
    };
```

Example modified from <https://gist.github.com/ehamberg/1263868>.

Scatterv

```
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);
```

```
sendcounts = (int*) malloc(sizeof(int)*size);
displs = (int*) malloc(sizeof(int)*size);
```

```
// calculate send counts and displacements
```

```
int rem = (N*N) % size;// elements remaining after division among processes
```

```
for (int i=0; i<size; i++) {
    sendcounts[i] = (N*N) / size;
    if (rem-- > 0)
        sendcounts[i]++;
    displs[i] = sum;
    sum += sendcounts[i];
}
```

Treat it as a 1D vector

```
// print calculated send counts and displacements for each process
```

```
if (0 == rank) {
    for (int i=0; i<size; i++)
        printf("sendcounts[%d] = %d\tdispls[%d] = %d\n", i, sendcounts[i],
            i, displs[i]);
}
```

```
// divide the data among processes as described by sendcounts and displs
```

```
MPI_Scatterv(&data, sendcounts, displs, MPI_CHAR,
    &rec_buf, 100, MPI_CHAR, 0, MPI_COMM_WORLD);
```


Scatterv

```
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);
```

```
sendcounts = (int*) malloc(sizeof(int)*size);
displs = (int*) malloc(sizeof(int)*size);
```

```
// calculate send counts and displacements
```

```
int rem = (N*N) % size;// elements remaining after division among processes
```

```
for (int i=0; i<size; i++) {
    sendcounts[i] = (N*N) / size;
    if (rem-- > 0)
        sendcounts[i]++;
    displs[i] = sum;
    sum += sendcounts[i];
}
```

Treat it as a 1D vector

Find displacements, some may need extra if $n \% p \neq 0$

```
// print calculated send counts and displacements for each process
```

```
if (0 == rank) {
    for (int i=0; i<size; i++)
        printf("sendcounts[%d] = %d\t displs[%d] = %d\n", i, sendcounts[i],
            i, displs[i]);
}
```

```
// divide the data among processes as described by sendcounts and displs
```

```
MPI_Scatterv(&data, sendcounts, displs, MPI_CHAR,
    &rec_buf, 100, MPI_CHAR, 0, MPI_COMM_WORLD);
```

Scatterv

```
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);
```

```
sendcounts = (int*) malloc(sizeof(int)*size);
displs = (int*) malloc(sizeof(int)*size);
```

```
// calculate send counts and displacements
```

```
int rem = (N*N) % size;// elements remaining after division among processes
```

```
for (int i=0; i<size; i++) {
    sendcounts[i] = (N*N) / size;
    if (rem-- > 0)
        sendcounts[i]++;
    displs[i] = sum;
    sum += sendcounts[i];
}
```

Treat it as a 1D vector

Find displacements, some may need extra if $n \% p \neq 0$

```
// print calculated send counts and displacements for each process
```

```
if (0 == rank) {
    for (int i=0; i<size; i++)
        printf("sendcounts[%d] = %d\t displs[%d] = %d\n", i, sendcounts[i],
            i, displs[i]);
}
```

Prefix sum of send counts

```
// divide the data among processes as described by sendcounts and displs
```

```
MPI_Scatterv(&data, sendcounts, displs, MPI_CHAR,
    &rec_buf, 100, MPI_CHAR, 0, MPI_COMM_WORLD);
```


Scatterv

```
// print what each process received
printf("%d: ", rank);
for (int i=0; i < sendcounts[rank]; i++) {
    printf("%c\t", rec_buf[i]);
}
printf("\n");

MPI_Finalize();

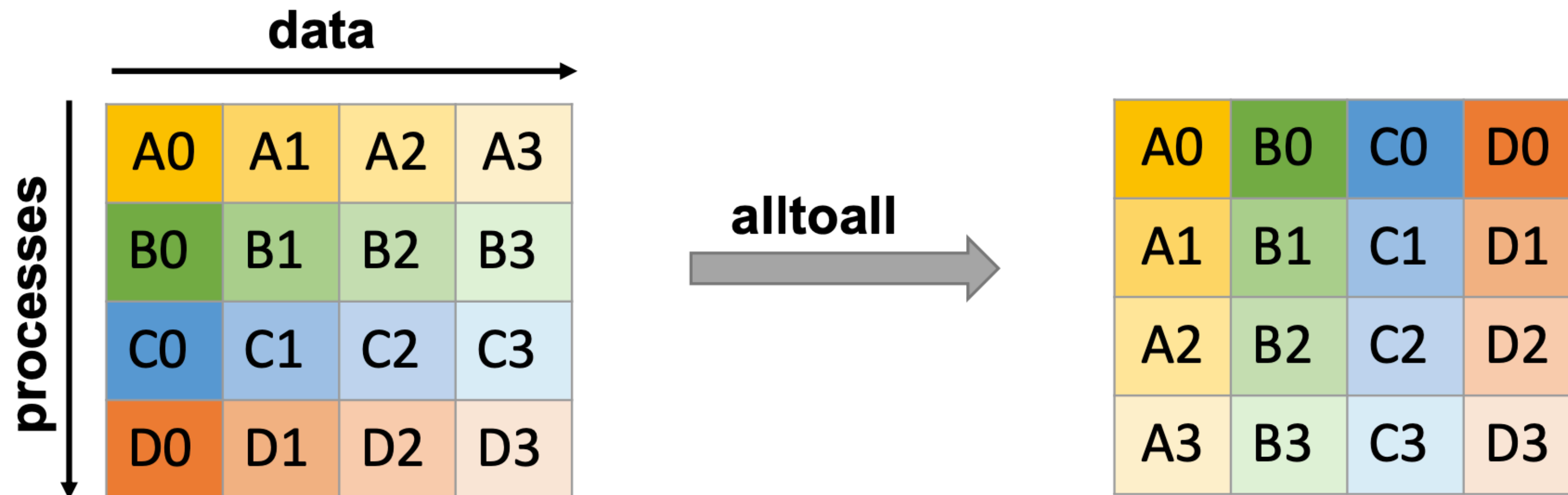
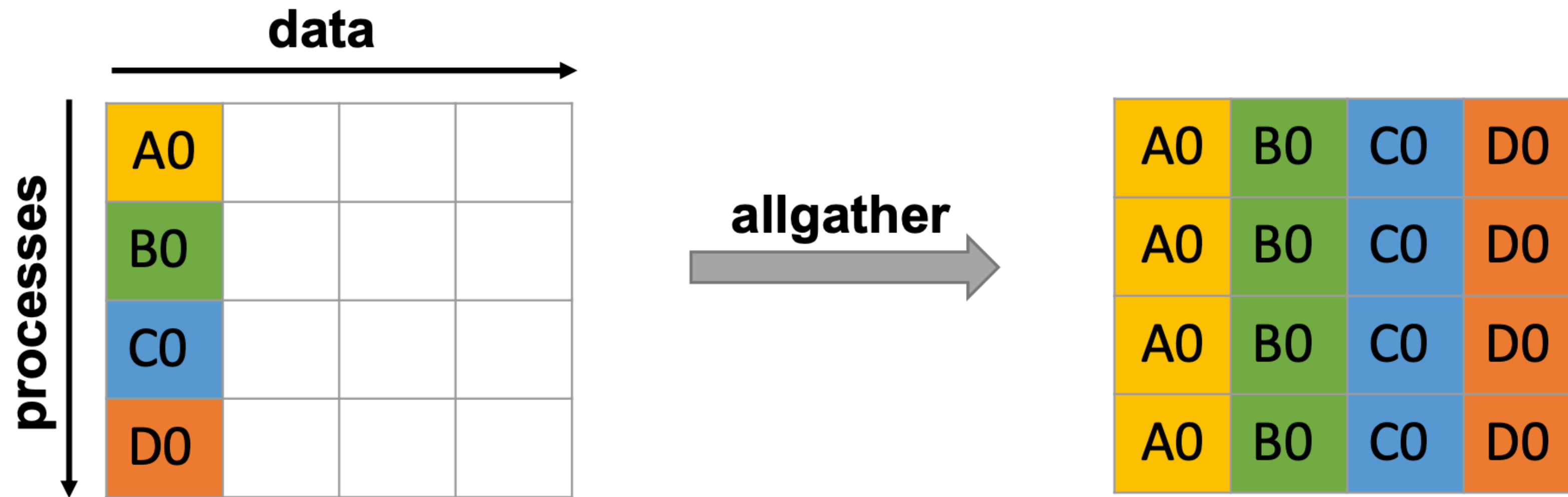
free(sendcounts);
free(displs);

return 0;
}
```

% mpirun -np 3 scatterv

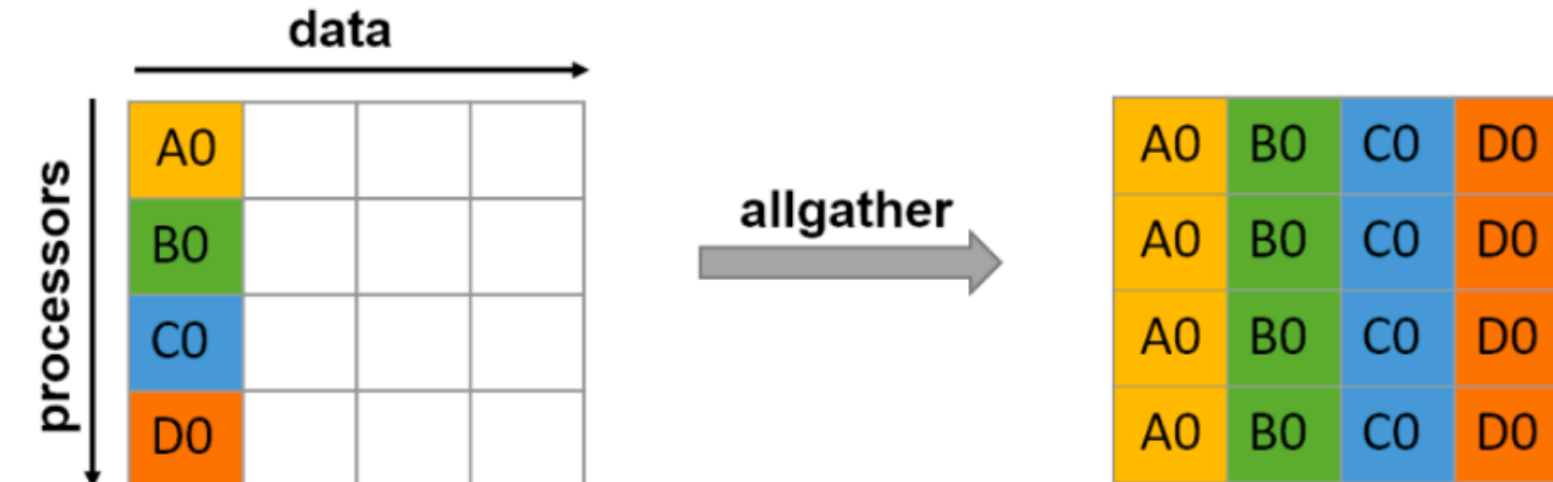
```
sendcounts[0] = 6 displs[0] = 0
sendcounts[1] = 5 displs[1] = 6
sendcounts[2] = 5 displs[2] = 11
0: a    b    c    d    e    f
1: g    h    i    j    k
2: l    m    n    o    p
```

Allgather, Alltoall



Allgather

Allgather: corresponds to each process broadcasting its data



```
int MPI_Allgather(void* sbuf, int scount, MPI_Datatype stype,
                  void* rbuf, int rcount, MPI_Datatype rtype,
                  MPI_Comm comm);
```

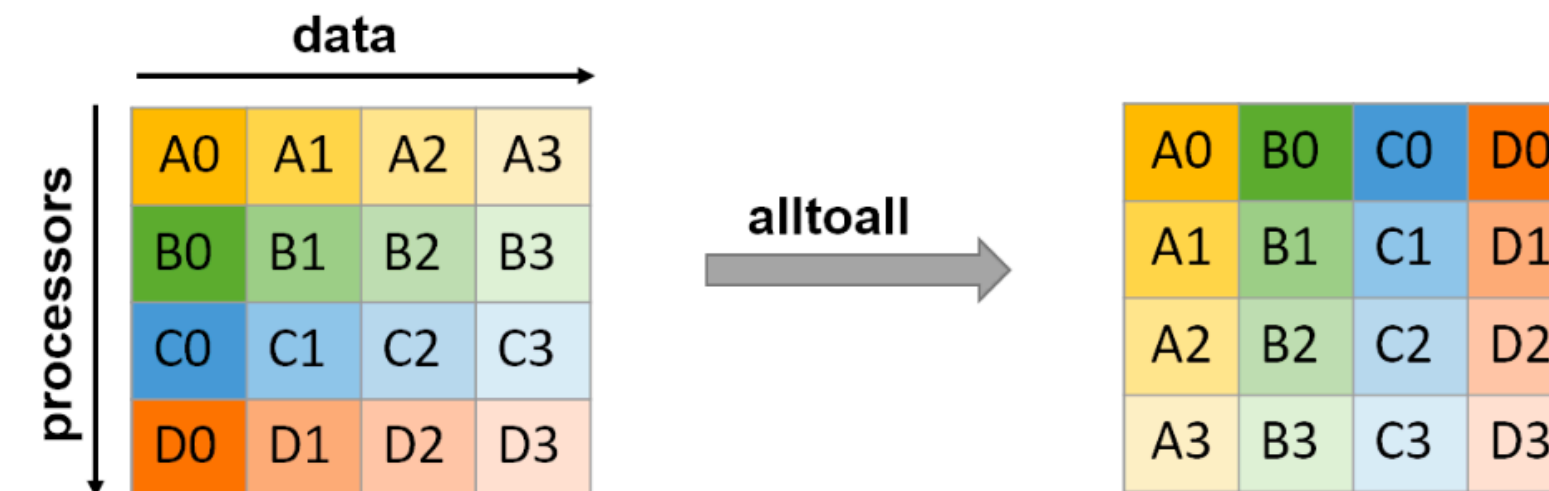
Example:

```
int main(int argc, char* argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    std::vector<int> buf;
    buf.resize(size, -1);
    MPI_Allgather(&rank, 1, MPI_INT,
                 &buf[0], 1, MPI_INT, MPI_COMM_WORLD);
    return MPI_Finalize();
}
```

At this point, what is
the content of the
vector buf?

Alltoall

Alltoall: every process has one message for every other process



```
int MPI_Alltoall(void* sbuf, int scount, MPI_Datatype stype,  
                void* rbuf, int rcount, MPI_Datatype rtype,  
                MPI_Comm comm);
```

```
int main(int argc, char* argv []) {  
    int rank, size;  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
    MPI_Comm_size(MPI_COMM_WORLD, &size);  
    std::vector<float> send(size);  
    std::vector<float> recv(size, -1);  
    for (int i = 0; i < size; ++i) send[i] = rank;  
    MPI_Alltoall(&send[0], 1, MPI_FLOAT,  
                &recv[0], 1, MPI_FLOAT, MPI_COMM_WORLD);  
    return MPI_Finalize();  
}
```

At this point, what is the content of the vector recv?

Collective Communication

Summary:

- All processes within a communicator must participate
- All collective operations are **blocking**
- Except of `MPI_Barrier`, no synchronization can be assumed
- All collective operations are guaranteed to not interfere with point-to-point messages
- Collective operations can be implemented using only point-to-point calls

Operations:

- barrier, broadcast, scatter, gather, allgather, alltoall

Reductions

- Let's assume we want to compute a sum over n elements:

```
int sum = 0;
for (int i = 0; i < n; ++i) sum = sum + A[i];
```

- Can we do this in parallel? -> Parallel Reductions

```
int MPI_Reduce(void* sbuf, void* rbuf, int count, MPI_Datatype type,
               MPI_Op op, int root, MPI_Comm comm);
```

Example:

```
// assume each processor has one element:
int a = A[rank];
int sum = 0;
MPI_Reduce(&a, &sum, 1, MPI_INT,
           MPI_SUM, 0, MPI_COMM_WORLD);
std::cout << sum << std::endl;
```

Reductions

- What about other binary operations?

```
int sum = A[0];  
for (int i = 1; i < n; ++i) sum = op(sum, A[i]);
```

- Can we do this in parallel?
 - only if **op(x,y)** is associative: $op(x, op(y, z)) == op(op(x, y), z)$
- MPI_Reduce takes the combination operator as MPI_Op:

MPI_Op	Meaning	C operator
MPI_MIN	minimum	$x < y ? x : y$
MPI_MAX	maximum	$x < y ? y : x$
MPI_SUM	sum	$x + y$
MPI_PROD	product	$x * y$
MPI_LAND, MPI_LOR	Logical and/or	$x \&\& y, x \ \ y$
MPI_BAND, MPI_BOR	Binary and/or	$x \& y, x \mid y$

Reductions

- `MPI_Allreduce = MPI_Reduce + MPI_Bcast`

```
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,  
                  MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

Example:

```
while (true) {  
    // compute ...  
    // check termination  
    int terminate = ...;  
    int result;  
    MPI_Allreduce(&terminate, &result, 1, MPI_INT,  
                  MPI_LAND, MPI_COMM_WORLD);  
    if (result == 1) break;  
}
```


Reductions

Reduction Example:

```
float A[p];
float master_sum = 0.0f;
if (rank == 0)
    for (int i = 0; i < p; ++i) {
        A[i] = 1.0f*rand()/RAND_MAX;
        master_sum += A[i];
    }

float value;
MPI_Scatter(A, 1, MPI_FLOAT, &value, 1,
           MPI_FLOAT, 0, MPI_COMM_WORLD);

float sum;
MPI_Reduce(&value, &sum, 1, MPI_FLOAT,
           MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0 && master_sum != sum) {
    printf("ERROR!\n");
}
```

What happens?

Generates array of random numbers and sums them up

Sends one random number to each process

Parallel sum over random numbers

Why might this print "Error!"?

Answer: floating point math is not associative!

Parallel Prefix

- Let's consider a prefix sum operation:

```
// prefix sum:  
int x[n], y[n];  
y[0] = x[0];  
for (int i = 1; i < n; ++i)  
    y[i] = op(y[i-1], x[i]);
```

- With MPI:

```
int MPI_Scan(void *sendbuf, void *recvbuf, int count,  
             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

Example:

```
// assume each processor has one element:  
  
int prefix = 0;  
MPI_Scan(&rank, &prefix, 1, MPI_INT,  
        MPI_SUM, MPI_COMM_WORLD);  
std::cout << prefix << std::endl;
```

Communicator Constructors

MPI provides functions to create a new communicator from an existing one:

Duplicate the same communicator

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm* newcomm);
```

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,  
MPI_Comm* newcomm);
```

Communicators can be released when no longer needed:

```
int MPI_Comm_free(MPI_Comm* comm);
```


Communicator Constructors

MPI provides functions to create a new communicator from an existing one:

Duplicate the same communicator

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm* newcomm);
```

```
int MPI_Comm_split(MPI_Comm comm, int color, int key,  
MPI_Comm* newcomm);
```

Each process gets a new communicator depending on color

All processes with the same color will belong to the same new communicator, ordered by key

Communicators can be released when no longer needed:

```
int MPI_Comm_free(MPI_Comm* comm);
```


Communicator Split Example

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int color = rank % 2;
    MPI_Comm comm;

    MPI_Comm_split(MPI_COMM_WORLD, color, rank, &comm);
    int S = 0;
    MPI_Reduce(&rank, &S, 1, MPI_INT, MPI_SUM, 0, comm);
    MPI_Comm_free(&comm);

    std::cout << rank << " " << S << std::endl;
    return MPI_Finalize();
}
```

Reduce on
even and
odd

New communicator

Communicator Split Example

$\text{MPI_WORLD} = \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_7\}$

	P_0	P_1	P_2	P_3	P_4	P_5	P_6	P_7
rank	0	1	2	3	4	5	6	7
color	0	1	0	1	0	1	0	1
key	0	1	2	3	4	5	6	7

$\text{comm} = \{P_0, P_2, P_4, P_6\} \text{ or } \{P_1, P_3, P_5, P_7\}$

	P_0	P_2	P_4	P_6	P_1	P_3	P_5	P_7
rank	0	1	2	3	0	1	2	3

Communicator Split Example

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[]) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    int color = rank % 2;
    MPI_Comm comm;

    MPI_Comm_split(MPI_COMM_WORLD, color, rank, &comm);
    int S = 0;
    MPI_Reduce(&rank, &S, 1, MPI_INT, MPI_SUM, 0, comm);
    MPI_Comm_free(&comm);

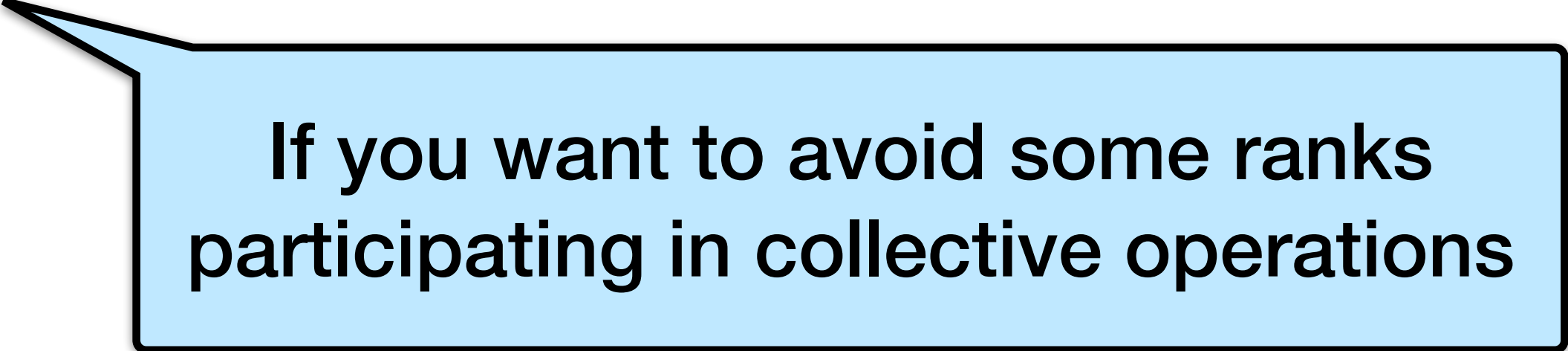
    std::cout << rank << " " << S << std::endl;
    return MPI_Finalize();
}
```

5	0
2	0
7	0
3	0
6	0
4	0
0	12
1	16

What is the output for p

Other Default Communicators

- Communicator constructors are collective operations
- MPI provides two additional predefined communicators
MPI_COMM_NULL and MPI_COMM_SELF



If you want to avoid some ranks participating in collective operations

MPI References

- MPI Standard: <https://www.mpi-forum.org/>
- Other information: <https://www.mcs.anl.gov/research/projects/mpi/index.htm>
- LLNL tutorial: <https://hpc-tutorials.llnl.gov/mpi/>
- <https://mpitutorial.com/>